

Les fonctions stockées MYSQL

Les fonctions Stockées

Définition :

Une **fonction stockée** dans MySQL est un ensemble d'instructions SQL qui exécutent une tâche/opération et renvoient une valeur unique.

Une fonction stockée est presque similaire à la procédure dans MySQL, mais elle présente quelques différences qui sont les suivantes :

- Le paramètre de fonction peut contenir uniquement le paramètre **IN** sans pouvoir spécifier ce type de paramètre. Par contre la procédure peut autoriser les paramètres **IN**, **OUT**, **INOUT**.
- La fonction stockée ne peut renvoyer qu'une seule valeur définie dans l'en-tête de la fonction.
- La fonction stockée peut également être appelée dans des instructions SQL.

Lorsque le but de notre programme est de calculer et de renvoyer une seule valeur alors on utilise les fonctions.

Création d'une fonction stockée :

La syntaxe suivante est utilisée pour créer une fonction stockée dans MySQL :

```
DROP FUNCTION IF EXISTS func_name;
DELIMITER $$
CREATE FUNCTION func_name(func_parameter(s))
RETURNS datatype
{characteristic }
BEGIN
    fun_body;
END
DELIMITER ;
```

La syntaxe de la fonction stockée utilise les paramètres suivants qui sont décrits ci-dessous :

- **func_name** : c'est le nom de la fonction stockée.
- **func_parameter** : il contient la liste des paramètres utilisés par le corps de la fonction. Il ne permet pas de spécifier les paramètres **IN**, **OUT**, **INOUT**.
- **datatype** : c'est un type de données de la valeur de retour de la fonction.
- **characteristics** : l'instruction CREATE FUNCTION n'est acceptée que lorsque les caractéristiques sont définies dans la déclaration. Ce champ peut prendre l'une des valeurs :
 - **DETERMINISTIC** : Une fonction déterministe produit toujours le même résultat pour les mêmes entrées. Cela signifie qu'une fonction donnée avec les mêmes arguments renverra toujours le même résultat.
 - **READS SQL DATA** : Indique que la fonction peut lire des données dans la base de données, mais ne les modifie pas. C'est typiquement utilisé pour des fonctions qui effectuent des opérations de lecture.

- **MODIFIES SQL DATA** : Indique que la fonction modifie des données dans la base de données. Cela peut inclure des opérations de mise à jour, d'insertion ou de suppression.
 - **CONTAINS SQL** : Signifie que la fonction peut contenir des instructions SQL arbitraires. Cela inclut à la fois la lecture et la modification des données.
 - **NO SQL** : Cela signifie que la fonction ne contient aucune instruction SQL. Elle est généralement utilisée pour des fonctions qui effectuent des calculs ou des opérations sans interagir directement avec la base de données.
- **fun_body** : un ensemble d'instructions SQL pour effectuer les opérations. Il nécessite au moins une instruction **RETURN**. Lorsque l'instruction return est exécutée, la fonction se termine automatiquement.

Remarque : on peut aussi commencer la déclaration de la fonction par la syntaxe :

```
CREATE FUNCTION [IF NOT EXISTS] func_name(...)
```

Appeler une fonction stockée

Une fonction stockée est appelée avec l'instruction SQL :

```
SELECT func_name(parameter(s))
```

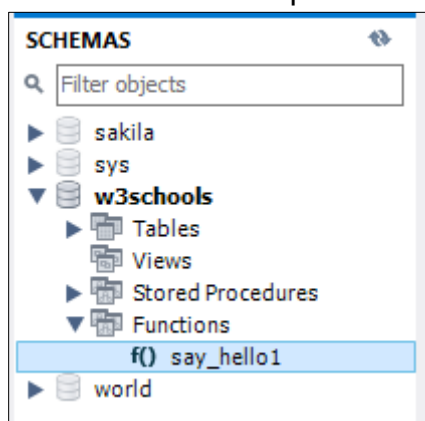
Exemple de fonction stockée sans paramètre

On veut créer une fonction stockée qui retourne au texte "Hello world" :

```
DELIMITER $$
CREATE FUNCTION say_hello1()
RETURNS VARCHAR(20)
NO SQL
BEGIN
    DECLARE msg VARCHAR(20);
    SET msg = "Hello world";
    RETURN msg;
END $$
DELIMITER ;

SELECT say_hello1();
```

Les fonctions créées par l'utilisateur se trouvent dans le dossier **Functions** :



On peut aussi écrire l'exemple précédent de la façon suivante :

```
DELIMITER $$
CREATE FUNCTION say_hello2()
RETURNS VARCHAR(20)
NO SQL
BEGIN
    RETURN "Hello world";
END $$
DELIMITER ;
SELECT say_hello2();
```

Exemple de fonction stockée sans paramètre

On veut créer une fonction stockée qui prend comme paramètre le prix d'un article et qui retourne la catégorie de cet article selon le tableau suivant :

Price	Category
price < 50	"Category A"
50 ≤ price < 100	"Category B"
price ≥ 100	"Category C"

Ensuite on va exploiter cette fonction pour afficher la catégorie de chaque article de la table **products** de la base de données **w3schools**.

```
DELIMITER $$
CREATE FUNCTION IF NOT EXISTS category_price(price DOUBLE)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE category VARCHAR(20);
    IF price < 50 THEN
        SET category = 'Category A';
    ELSEIF price < 100 THEN
        SET category = 'Category B';
    ELSE
        SET category = 'Category C';
    END IF;
    RETURN category;
END $$
DELIMITER ;

SELECT ProductID, ProductName, Price, category_price(Price) FROM products;
```

Exemple d'appel de fonction stockée dans la procédure

On veut créer une procédure stockée qui affiche les articles de la table **products** par ordre décroissant du champ **price** et qui invoque aussi la fonction **category_price** pour afficher la catégorie de chaque article :

```
DELIMITER $$
CREATE PROCEDURE IF NOT EXISTS get_productsDetails()
BEGIN
    SELECT ProductID, ProductName, Price, category_price(Price)
    FROM products
    ORDER BY Price DESC;
END $$
DELIMITER ;

CALL get_productsDetails();
```

Lister les fonctions stockées dans MySQL

Lorsque nous avons plusieurs fonctions dans le serveur MySQL, il est très important de lister toutes les fonctions. Car parfois les noms de fonction sont les mêmes dans de nombreuses bases de données.

Nous pouvons lister toutes les fonctions stockées sur le serveur MySQL actuel comme suit :

```
SHOW FUNCTION STATUS;
```

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
► sakila	get_customer_balance	FUNCTION	root@localhost	2023-01-02 16:27:40	2023-01-02 16:27:40	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sakila	inventory_held_by_cust...	FUNCTION	root@localhost	2023-01-02 16:27:40	2023-01-02 16:27:40	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sakila	inventory_in_stock	FUNCTION	root@localhost	2023-01-02 16:27:40	2023-01-02 16:27:40	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sys	extract_schema_from_fil...	FUNCTION	mysql.sys@local...	2023-01-02 16:21:13	2023-01-02 16:21:13	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sys	extract_table_from_file_...	FUNCTION	mysql.sys@local...	2023-01-02 16:21:13	2023-01-02 16:21:13	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sys	format_bytes	FUNCTION	mysql.sys@local...	2023-01-02 16:21:13	2023-01-02 16:21:13	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sys	format_path	FUNCTION	mysql.sys@local...	2023-01-02 16:21:13	2023-01-02 16:21:13	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
sys	format_statement	FUNCTION	mysql.sys@local...	2023-01-02 16:21:13	2023-01-02 16:21:13	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...

Pour afficher les fonctions stockées relative à une base de données spécifique :

```
SHOW FUNCTION STATUS WHERE Db = 'w3schools';
SHOW FUNCTION STATUS WHERE Db LIKE '%w3%';
```

On peut Aussi lister la liste des fonctions stockées en utilisant la table routine à l'aide de la syntaxe suivante :

```
SELECT
    routine_name
FROM
    information_schema.routines
WHERE
    routine_type = 'FUNCTION'
    AND routine_schema = 'database_name';
```

Exemple :

```
SELECT routine_name FROM information_schema.routines
WHERE routine_type = 'FUNCTION' AND routine_schema = 'w3schools';
```

Supprimer une fonction stockée :

Pour supprimer une fonction stockée, utiliser la syntaxe suivante :

```
DROP FUNCTION [IF EXISTS] func_name;
```

Exemple :

```
DROP FUNCTION say_hello1;
```